

🔥 Correct. 9/10.

You got the two things that matter:

- ☒ Read
- ☒ Write
- ☒ Execute
- ☒ Permissions provide **access control/authorization**
- ☒ They help protect files from unauthorized access or modification

A good mental model is:

File

↓

Who can access it?

↓

What can they do?

- └ Read
- └ Write
- └ Execute

For example:

```
secret.txt
User A → read + write
User B → read only
User C → no access
```

The OS enforces these permissions.

Tiny distinction

Authentication = *Who are you?*

Authorization = *What are you allowed to do?*

File permissions are primarily about **authorization**.



Round 19 — System Calls: Final Connection

We've already discussed system calls, but now I want you to connect **everything** we've learned.

Suppose your program wants to read:

```
secret.txt
```

Explain the high-level journey:

Application → ? → ? → **File** → ?

Include **user mode**, **kernel mode**, **system call**, and **permissions** in your explanation.

Don't worry about exact implementation details.

Chalo, connect the whole story. 🤖🔥



you click secret.txt so it signals os to open text editor -> which then in kernel mode opens the application along with the file -> user mode -> file editor in the ui

You're thinking in the right direction, but you've mixed up **opening a file through a GUI** with **the OS handling a file-read request**.

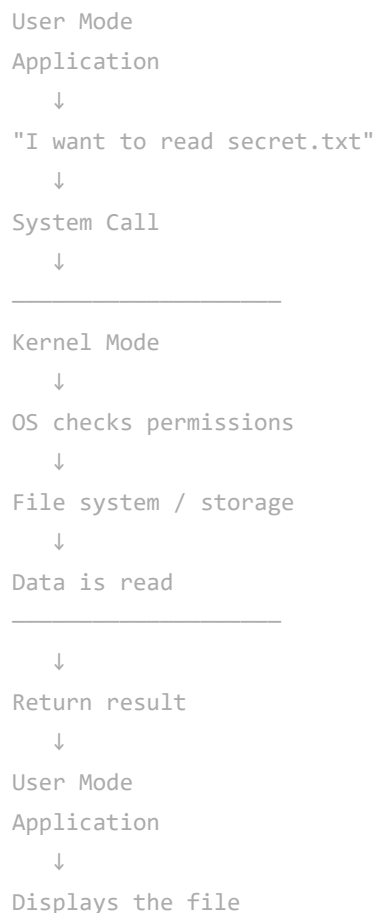
I'd give this **6.5/10** — the concepts are there, but let's tighten the flow.

What needs correction

When you click `secret.txt`, the OS may indeed help launch/open an appropriate application. But **the application itself normally runs in user mode**.

The kernel doesn't "open the text editor in kernel mode."

Instead, think about the actual **file-read operation**:



The key distinction

Text editor → user mode

OS kernel → kernel mode

System call → controlled doorway between them

Permissions → kernel checks whether the operation is allowed

So if the application asks:

"Can I read `secret.txt` ?"

The kernel effectively handles the protected operation and returns the result.

Your Day 2 mental model is getting strong

Look at how much we've connected already:

Program

↓

Process

↓

PCB

↓

Process States

↓

Threads

↓

Concurrency / Parallelism

↓

Context Switching

↓

CPU Scheduling

↓

Stack / Heap

↓

Virtual Memory

↓

File Permissions

↓

System Calls

↓

User Mode ↔ Kernel Mode

That's the OS foundation you actually want.

🔥 Final challenge for this section

Imagine an interviewer asks:

"What happens when you run a program?"

Don't give me a textbook answer.

Use **everything** you've learned today and explain the journey from:

Program on storage → OS → Process → Memory → CPU → execution

You can include threads, scheduling, system calls, etc. where they naturally fit.

Take your time.

This is your Day 2 boss fight. 🤖🔥

📄 🗣️ ...

when you run a program the
OS creates a process
OS goes to kernel mode and opens the file according to
privileges and system calls
and then the instructions are loaded through the memory
heirarchy and then the
process is in ready state once it turn comes it moves to
cpu and vecomes running and then it completes its
exeuction and finishes the state to terminated and empties
the call stack if its the single program running or simply
removes it from call stack



ChatGPT can make mistakes. Check important info.